



Scalable Patent Analysis Using Function Calling with Local Large Language Models

Juan José García Mesa
RTO Computational Research Accelerator
Arizona State University
Tempe, AZ, AZ, USA
juanjo.garciamesa@asu.edu

Abstract

User requests often foment technological innovation, prompting transformative change across research and functional areas. At Arizona State University, a collaboration with their intellectual property management services led to the development of an automated tool that addresses the growing need for efficient patent classification and analysis. The system leverages large language models integrated with function calling to retrieve, clean, and analyze patent data from the US Patent and Trademark Office data portal. Key functionalities include extracting university-assigned patents, classifying them using a streamlined version of the official code hierarchy, and executing targeted queries such as retrieving detailed patent information or identifying patents related to specific subject matters. This work contributes to more efficient patent management processes and serves as a blueprint for applying similar methodologies to related problems. This approach combines conventional data analysis with dynamic function calling, mitigating common artificial intelligence challenges such as hallucinations and providing accurate outputs.

CCS Concepts

• **Software and its engineering** → **Software design engineering**.

Keywords

Large language models, function calling, patent classification

ACM Reference Format:

Juan José García Mesa. 2025. Scalable Patent Analysis Using Function Calling with Local Large Language Models. In *Practice and Experience in Advanced Research Computing (PEARC '25)*, July 20–24, 2025, Columbus, OH, USA. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3708035.3736071>

1 Introduction

Patent classification and analysis present significant challenges for research institutions such as Arizona State University (ASU). This work addresses these challenges by developing a tool that streamlines the retrieval, classification, and querying of patent data,

tailored to the needs at ASU. A prime example is a challenge from Skysong Innovations (SI), which manages intellectual property at ASU and frequently encounters complex queries about its portfolio of inventions, ranging from general questions about patent classification by technical field to domain-specific inquiries.

Building on previous work at the university — which leveraged an efficient framework for deploying locally operated large language models (LLMs) [5] for data retrieval, information summarization, and text generation [2] — I developed a solution to streamline this manual process. This approach integrates semantic similarity search, the natural language parsing and decision-making capabilities of LLMs, and conventional data analysis techniques to retrieve and examine patent data from the US Patent and Trademark Office (USPTO) data portal.

2 Motivation

This collaboration with SI began at an outreach event showcasing ASU's advanced computational infrastructure and capabilities, focusing on generative artificial intelligence (AI) tools. Following the event, the center contacted us with a request for a tool to efficiently address internal and external inquiries regarding the thousands of patents it manages. The original request emphasized the growing need to summarize and analyze ASU's extensive patent portfolio in an automated fashion. They explained that the tool should be capable of automatically downloading accepted patent and patent application data from the USPTO data portal and then generating a database to answer queries using a large language model. In practice, users should be able to ask for summaries or the composition of ASU's issued patents by technical subject matter in plain English, obtain a list of patents and patent applications by number and title under each technical category, or receive tailored outputs such as a detailed list of technologies related to a specific innovation focus.

Given this challenge, the proposed solution combines open-source models, function calling, and semantic similarity search to make patent retrieval, classification, and summarization more efficient. The approach also opens the door for adapting similar methods to related tasks.

3 Implementation

3.1 Patents Retrieval & Filtering

An essential step in the project was filtering the patents assigned to ASU from the extensive USPTO database. The data portal offers 32 different types of bulk data related to patents. One of them, product identifier PTGRXML, has been released every Tuesday since January 2002, containing the full text of each patent granted

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
PEARC '25, Columbus, OH, USA

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1398-9/25/07
<https://doi.org/10.1145/3708035.3736071>

in that week. Released in extensible markup language (XML) format, each of the approximately 1,200 releases contained hundreds of patents, with only a few assigned to ASU, making it essential to extract the specific records for efficiency. Given the number and size of the bulk files, the automated process used the available REST API to download each file, extract the ASU-related patents, and discard the remainder to conserve storage space. Each unzipped XML file size was between 0.5 and 1 GiB in size and contained around 2,000 patents on average.

Each patent entry was a complete XML document, spanning between 700 and 2000 lines in the XML file. Based on this structure, the parsing function iterated through the patent XML documents on each file to extract the ASU-related patents. The system identified relevant patents by filtering for ASU variations in the assignee organization tag, such as "Arizona State University," "Arizona State," and the "Arizona Board of Regents on Behalf of Arizona State University."

3.2 Data Extraction

The XML structure presented significant challenges due to its complexity and lack of dedicated parsing libraries. While an available 116-page document describes the US Patent Grand Data XML v4.2 (Reed Book version) and helped identify tags of interest, the data extraction was still filled with intricacies. Each XML file consisted of a collection of concatenated XML documents laden with extensive metadata, including multiple dates (e.g., date produced, application date, publication date), a long list of examiners, and convoluted nested structures that included parent and child elements along with various formatting metacharacters.

To tackle these challenges, every key element - such as patent title, publication date, publication number, cooperative patent classification code (CPC), abstract, and full description - was carefully extracted and saved into individual files, resulting in data for approximately 1,400 patents. The primary Python library for navigating the XML structure and data extraction was 'etree' from 'lxml,' used to navigate the convoluted XML hierarchy and find the relevant tags. Despite the complexities, this process ensured that the required patent data was adequately structured for the subsequent analysis.

3.3 Data Analysis

The USPTO uses the CPC codes to categorize and organize patent documents based on their technological content. The code system is an extension of the International Patent Classification jointly managed by the USPTO and the European Patent Office. The codes are hierarchical and divided into sections A through Y (omitting letters I to X), subdivided into classes, subclasses, groups, and subgroups (e.g., Fig. 1). Initially, the five-level hierarchical list of codes had over 260,000 entries, making it overly specific for the database of ASU patents. Reducing them to 824 three-level codes representing section, class, and subclass levels simplified the classification and served as a concise label for patent categorization.

The various analytical needs required a system with several functionalities. The application can retrieve patent information by searching patents based on publication number or classify the entire patent portfolio by section, returning counts for each category.

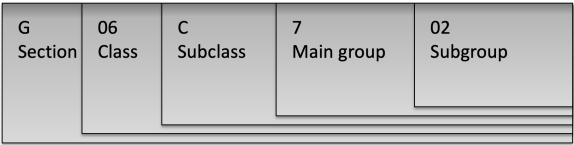


Figure 1: Example of a Cooperative Patent Classification code. Five level hierarchical structure of the CPC code G06C7/02, specifically under group G “physics”, class 06 “computing; calculating or counting”, subclass C “digital computers in which all the computation is effected mechanically”, group 7 “input mechanisms”, and subgroup 02 “keyboards”.

In addition, users can identify patents related to a specific field (e.g., sustainability) by providing a term, and the system lists the patents that fall under the most similar CPC category to that area. Another feature allows users to search for patents by title or abstract, effectively finding entries that contain a given query string. Finally, the last implemented option allows the user to specify a section or class and list the patent counts organized by class or subclass, respectively.

Among these functionalities, finding patents related to a field proved to be the most challenging. The difficulty arises when the fields defined by user queries differ from the technical subject matters delineated by the CPC codes. The application performs semantic similarity search by applying vector similarity algorithms using the ChromaDB library, an approach commonly used for retrieving relevant information from a vector database in retrieval-augmented generation. This method computes the distance between the embeddings of the user-provided query and the CPC descriptors using the squared L2 distance to identify the most relevant section, class, or subclass in the embedding space. Finally, a graphical user interface built with the Python library Gradio [1] provides an accessible means for users to interact with these functionalities (Fig. 2).

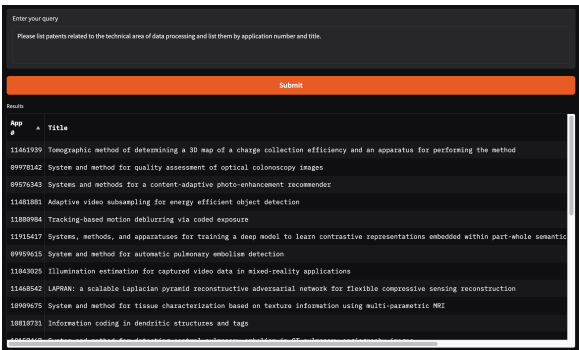


Figure 2: Patent search interface. The interface allows users to enter queries and use the functionalities described in the text. This figure showcases an example where patents related to a specific field, such as “data processing” are retrieved. The results are shown below, in this case with application number and title information, as requested in the input.

3.4 Function Calling

The SI team was interested in an interface where they could enter natural language as input for their queries instead of selecting filters and parameter values. The tool leverages a local LLM to interpret natural language input and select the appropriate function to call via a function-calling interface, enabling dynamic and flexible interactions. While function calling is the mechanism that allows the LLM to interact with external tools, it is the LLM that interprets user intent and determines which function is best suited for the query. This approach transforms LLMs from static text generators into dynamic assistants that can interact with external tools, APIs, and custom-defined functions. When a user submits a query, the LLM analyzes it to determine whether it requires an external action. If so, the model selects the appropriate function from a given set, extracts the values for the required parameters, and executes the call. The output is integrated into a final response, enabling the LLM to perform tasks beyond text generation and significantly broadening its applications.

In this implementation, the system defines a collection of tools as a JSON list, where each tool represents an available function with its name, description, and argument schema. For example, functions such as `find_similar_patents_by_title` and `classify_patents_by_section` include detailed argument specifications to ensure the LLM knows what parameter values to generate. The process begins with a crafted system prompt instructing the LLM to analyze the user query that, if applicable, returns a function call in a strict format. The expected output format is a concise list of function calls in the form `func_name1(param1=val1, ...), func_name2(params)`. This explicit instruction minimizes ambiguity and ensures the LLM returns a directly actionable response.

The core function orchestrates the end-to-end process, constructing a conversation with two messages, one containing the system prompt, which includes the list of tools, and one with the user query. These messages are tokenized and fed into the LLM, which generates a response containing the function call. Then, the response is run through the helper function so that Python can dynamically invoke the tool call. This additional step ensures the robustness of the process by taking a list of argument strings formatted as value and key pairs and converting them into a Python dictionary. When applicable, it also performs type conversions, such as casting strings to booleans or integers.

LLMs are evaluated using various benchmarks to assess their performance on different tasks. One of these benchmarks is the Berkeley function calling leaderboard [7], which ranks models based on their ability to call functions accurately. At the time of development, the Watt Tool model occupied the top position on the leaderboard and, therefore, was used in this project. This LLM was explicitly designed to excel at complex tool usage scenarios that require multi-turn interactions, and the 8b version fine-tunes the Llama 3.1 8b instruct model [4] using supervised learning on a specialized dataset, as described in [6]. A locally hosted Watt Tool 8b model instance is integrated into the application, enabling reliable function execution.

4 Conclusion

This project demonstrates how LLM-driven function calling can transform complex data retrieval and classification tasks into a streamlined process that supports the intellectual property department at Arizona State University. By leveraging a local LLM to interpret user intent and select the most appropriate function to call, the application enables dynamic, natural language-driven workflows for patent analysis. Function calling serves as the mechanism for the LLM to interact with external tools, but it is the LLM's reasoning that allows for flexible and context-aware responses. The application can improve operational efficiency by automating the extraction of patent data from the USPTO portal, cleaning and organizing it, and providing a user-friendly interface. Beyond addressing the specific challenges of patent analysis, this work serves as a blueprint for extending similar methodologies to related classification problems. For instance, in domains where predefined codes or labels guide data organization (e.g., CPC codes for patents), integrating LLM-driven function calling can help reduce hallucinations that often plague generative AI results due to misclassification, misinterpretation, or out-of-context errors. Integrating natural language processing with traditional data extraction provides a robust framework for enhancing accuracy and reliability in data analysis.

Future work includes leveraging a newly launched USPTO data portal with a more robust API and JSON-formatted outputs, simplifying data retrieval and cleaning. In addition, addressing the limitation of the sometimes ambiguous mapping of user-defined categories to CPC labels remains a priority, as does developing an automated system (e.g., a daily Slurm cron job) to keep the patent database continuously updated. A comprehensive evaluation of the effectiveness of the tools is also planned, including user studies, quantitative analysis of search accuracy and efficiency, and systematic collection of feedback from researchers. Together, these advancements promise to extend the utility of the current approach.

5 Code Availability

The source code for the project is freely available on the GitHub repository
<https://github.com/jgarciamesa/patent-analysis>.

Acknowledgments

The author acknowledges Skysong Innovations, especially Jordan Bell and Kyle Siegal, for initiating this collaboration and their trust in his ability to develop a tool that supports their needs. Their engagement and insights played a key role in shaping this project.

The author acknowledges Research Computing at Arizona State University for providing HPC and storage resources [3].

The author acknowledges four anonymous reviewers for their helpful comments on earlier versions of this manuscript.

References

- [1] Abubakar Abid, Ali Abdalla, Ali Abid, Dawood Khan, Abdulrahman Alfozan, and James Y. Zou. 2019. Gradio: Hassle-Free Sharing and Testing of ML Models in the Wild. *CoRR abs/1906.02569* (2019). arXiv:1906.02569 <http://arxiv.org/abs/1906.02569>
- [2] Juan Jose Garcia Mesa and Gil Speyer. 2024. Enhancing the application of large language models with retrieval-augmented generation for a research community. doi:10.5281/zenodo.13368208

- [3] Douglas M. Jennewein, Johnathan Lee, Chris Kurtz, Will Dizon, Ian Shaeffer, Alan Chapman, Alejandro Chiquete, Josh Burks, Amber Carlson, Natalie Mason, Arhat Kobwala, Thirugnanam Jagadeesan, Praful Barghav, Torey Battelle, Rebecca Belshe, Debra McCaffrey, Marisa Brazil, Chaitanya Inumella, Kirby Kuznia, Jade Buzinski, Sean Dudley, Dhruvil Shah, Gil Speyer, and Jason Yalim. 2023. The Sol Supercomputer at Arizona State University. In *Practice and Experience in Advanced Research Computing* (Portland, OR, USA) (PEARC '23). Association for Computing Machinery, New York, NY, USA, 5 pages. doi:10.1145/3569951.3597573
- [4] David Patterson, Joseph Gonzalez, Urs Hölzle, Quoc Le, Chen Liang, Lluís-Miquel Munguia, Daniel Rothchild, David So, Maud Texier, and Jeff Dean. 2022. The Carbon Footprint of Machine Learning Training Will Plateau, Then Shrink. arXiv:2204.05149 [cs.LG] <https://arxiv.org/abs/2204.05149>
- [5] Dhruvil Shah, Gil Speyer, and Jason Yalim. 2023. Centralized provisioning of large language models for a research community. In *Proceedings of the SC '23 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis* (Denver, CO, USA) (SC-W '23). Association for Computing Machinery, New York, NY, USA, 704–707. doi:10.1145/3624062.3624147
- [6] Wentao Shi, Mengqi Yuan, Junkang Wu, Qifan Wang, and Fuli Feng. 2025. Direct Multi-Turn Preference Optimization for Language Agents. arXiv:2406.14868 [cs.CL] <https://arxiv.org/abs/2406.14868>
- [7] Fanjia Yan, Huanzhi Mao, Charlie Cheng-Jie Ji, Tianjun Zhang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2024. Berkeley Function Calling Leaderboard. https://gorilla.cs.berkeley.edu/blogs/8_berkeley_function_calling_leaderboard.html.